

UNIVERSITÉ ROI HENRI CHRISTOPHE

Faculté des Sciences Informatiques — Niveau IV

Projet réalisé dans le cadre du cours de Programmation Python I
Enseigné par le Professeur Junior Semerizier

Date de remise : Mars 2026

Dépôt GitHub : github.com/borgellarouguessstyvespaul-collab/clinique-api

Membres du groupe :

BORGELLA Roguess Styves Paul
PERAD Sincy
MARC Calixte

TABLE DES MATIÈRES

1. Introduction
2. Mise en place de l'environnement de développement
3. Configuration de la base de données — database.py
4. Modélisation des données — models.py
 - 4.1 La table pivot Many-to-Many
 - 4.2 Les modèles Medecin, Patient, DossierMedical
 - 4.3 Les modèles Consultation et Medicament
5. Schémas de validation — schemas.py
6. Implémentation des endpoints — main.py
 - 6.1 Endpoints Médecins
 - 6.2 Endpoints Patients et Dossiers
 - 6.3 Endpoints Consultations
 - 6.4 Endpoints Prescriptions et Médicaments
7. Implémentation des relations SQLAlchemy
 - 7.1 One-to-One : Patient — DossierMedical
 - 7.2 One-to-Many : Medecin/Patient — Consultation
 - 7.3 Many-to-Many : Consultation — Medicament
8. Gestion des erreurs HTTP
9. Tests et validation via Swagger UI
10. Difficultés rencontrées et solutions apportées
11. Conclusion

1. INTRODUCTION

Ce projet est né d'une consigne simple : développer une API REST pour simuler le système d'information d'une clinique médicale. En apparence, ça semblait faisable. Mais une fois qu'on a commencé à lire les exigences — trois types de relations, vingt-deux endpoints, gestion des erreurs, tests Swagger — on a vite réalisé que ce n'était pas un simple exercice.

Notre groupe a choisi FastAPI comme framework principal, et ce choix s'est révélé excellent. FastAPI génère automatiquement une documentation interactive (Swagger UI) accessible sur /docs, ce qui nous a permis de tester chaque endpoint sans avoir besoin d'un outil externe. Pour la base de données, nous avons utilisé SQLite via SQLAlchemy ORM, ce qui signifie que toute notre interaction avec la base se fait en Python pur, sans écrire une seule ligne de SQL. Pydantic s'occupe de la validation des données entrantes, et Uvicorn sert de serveur ASGI pour exécuter l'application.

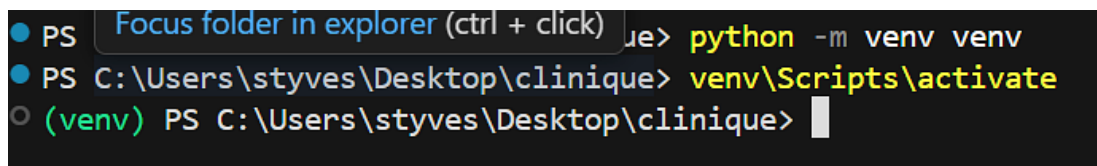
Le système gère quatre entités principales : les médecins (avec leur spécialité), les patients (avec leur dossier médical), les consultations et les médicaments. Entre ces entités, nous avons dû implémenter les trois types de relations fondamentales en base de données. Un patient a un seul dossier médical (One-to-One). Un médecin ou un patient peut avoir plusieurs consultations (One-to-Many). Et une consultation peut avoir plusieurs médicaments prescrits, un médicament pouvant apparaître dans plusieurs consultations (Many-to-Many).

2. MISE EN PLACE DE L'ENVIRONNEMENT DE DÉVELOPPEMENT

Avant d'écrire la moindre ligne de code, nous avons dû préparer l'environnement de travail. En Python, la bonne pratique est de créer un environnement virtuel pour isoler les dépendances de chaque projet. Sans ça, les bibliothèques s'installent globalement et ça crée des conflits de versions entre projets. Nous avons donc commencé par :

```
mkdir clinique cd clinique python -m venv venv venv\Scripts\activate
```

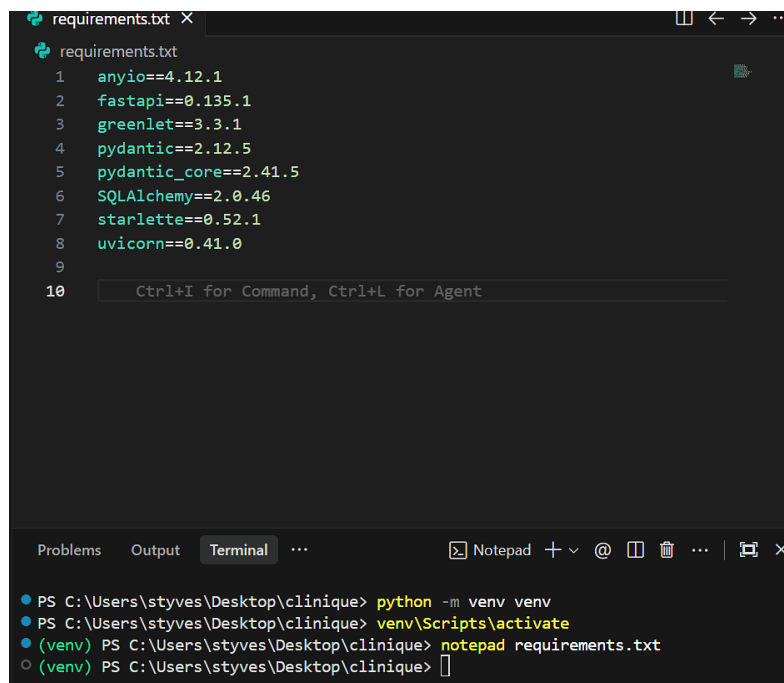
La commande `python -m venv venv` crée un dossier `venv/` contenant une installation Python isolée. L'activation avec `venv\Scripts\activate` fait apparaître le préfixe `(venv)` devant le prompt — c'est le signe qu'on travaille bien dans l'environnement isolé.



```
PS Focus folder in explorer (ctrl + click) > python -m venv venv
PS C:\Users\styves\Desktop\clinique> venv\Scripts\activate
(venv) PS C:\Users\styves\Desktop\clinique>
```

Figure 1 — Création et activation de l'environnement virtuel Python

Ensuite, nous avons créé le fichier `requirements.txt` qui liste toutes les bibliothèques nécessaires. Ce fichier est pratique parce que n'importe quel membre de l'équipe peut recréer exactement le même environnement avec une seule commande.



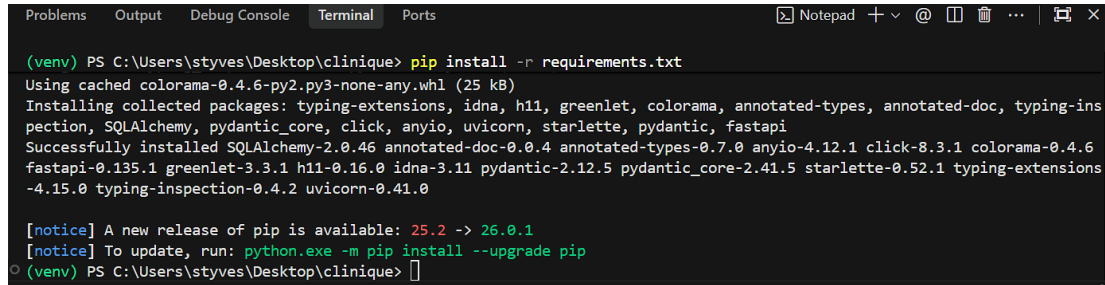
```
requirements.txt
1 anyio==4.12.1
2 fastapi==0.135.1
3 greenlet==3.3.1
4 pydantic==2.12.5
5 pydantic_core==2.41.5
6 SQLAlchemy==2.0.46
7 starlette==0.52.1
8 uvicorn==0.41.0
9
10 Ctrl+I for Command, Ctrl+L for Agent

Terminal
PS C:\Users\styves\Desktop\clinique> python -m venv venv
PS C:\Users\styves\Desktop\clinique> venv\Scripts\activate
(venv) PS C:\Users\styves\Desktop\clinique> notepad requirements.txt
(venv) PS C:\Users\styves\Desktop\clinique>
```

Figure 2 — Contenu du fichier `requirements.txt` avec les dépendances exactes

Nous avons ensuite installé toutes les dépendances avec `pip install -r requirements.txt`. SQLAlchemy, FastAPI, Pydantic, Uvicorn et leurs dépendances transitives s'installent

automatiquement.

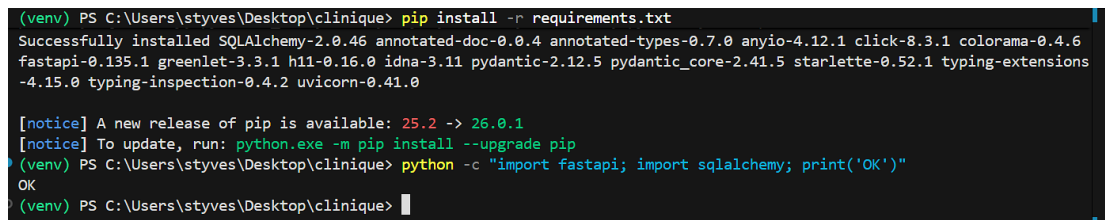


```
(venv) PS C:\Users\styves\Desktop\clinique> pip install -r requirements.txt
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: typing-extensions, idna, h11, greenlet, colorama, annotated-types, annotated-doc, typing-inspection, SQLAlchemy, pydantic_core, click, anyio, uvicorn, starlette, pydantic, fastapi
Successfully installed SQLAlchemy-2.0.46 annotated-doc-0.0.4 annotated-types-0.7.0 anyio-4.12.1 click-8.3.1 colorama-0.4.6 fastapi-0.135.1 greenlet-3.3.1 h11-0.16.0 idna-3.11 pydantic-2.12.5 pydantic_core-2.41.5 starlette-0.52.1 typing-extensions-4.15.0 typing-inspection-0.4.2 uvicorn-0.41.0

[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS C:\Users\styves\Desktop\clinique>
```

Figure 3 — Installation réussie de toutes les dépendances

Pour vérifier que tout fonctionnait, nous avons tapé la commande `python -c "import fastapi; import sqlalchemy; print('OK')"` qui retourne OK si les deux bibliothèques principales sont disponibles.

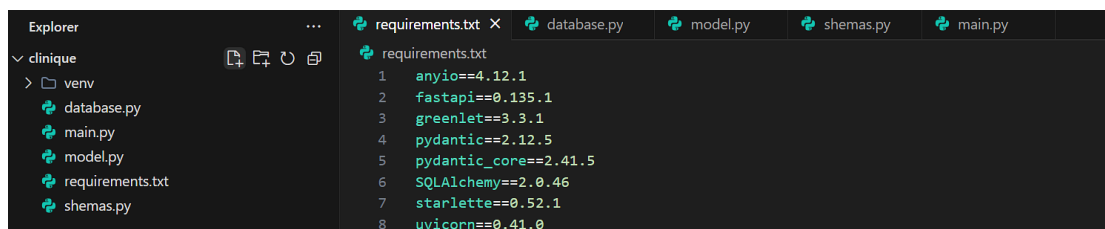


```
(venv) PS C:\Users\styves\Desktop\clinique> pip install -r requirements.txt
Successfully installed SQLAlchemy-2.0.46 annotated-doc-0.0.4 annotated-types-0.7.0 anyio-4.12.1 click-8.3.1 colorama-0.4.6 fastapi-0.135.1 greenlet-3.3.1 h11-0.16.0 idna-3.11 pydantic-2.12.5 pydantic_core-2.41.5 starlette-0.52.1 typing-extensions-4.15.0 typing-inspection-0.4.2 uvicorn-0.41.0

[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS C:\Users\styves\Desktop\clinique> python -c "import fastapi; import sqlalchemy; print('OK')"
OK
(venv) PS C:\Users\styves\Desktop\clinique>
```

Figure 4 — Vérification des installations : FastAPI et SQLAlchemy importés avec succès

La structure finale du projet comprend quatre fichiers Python principaux : `database.py` pour la connexion, `models.py` pour les tables, `schemas.py` pour la validation, et `main.py` pour les endpoints. Chaque fichier a une responsabilité unique et clairement définie.



```
Explorer
└─ clinique
  └─ venv
    ├── database.py
    ├── main.py
    ├── model.py
    ├── requirements.txt
    └─ schemas.py

requirements.txt
1 anyio==4.12.1
2 fastapi==0.135.1
3 greenlet==3.3.1
4 pydantic==2.12.5
5 pydantic_core==2.41.5
6 SQLAlchemy==2.0.46
7 starlette==0.52.1
8 uvicorn==0.41.0
```

Figure 5 — Structure finale du projet clinique dans VS Code

3. CONFIGURATION DE LA BASE DE DONNÉES — database.py

Le fichier database.py est court — une vingtaine de lignes — mais absolument fondamental. C'est lui qui établit le pont entre Python et SQLite, et qui fournit à FastAPI la mécanique nécessaire pour gérer les connexions à la base.

```
from sqlalchemy import create_engine from sqlalchemy.ext.declarative import
declarative_base from sqlalchemy.orm import sessionmaker DATABASE_URL =
"sqlite:///./clinique.db" engine = create_engine( DATABASE_URL,
connect_args={"check_same_thread": False} ) SessionLocal =
sessionmaker(autocommit=False, autoflush=False, bind=engine) Base =
declarative_base() def get_db(): db = SessionLocal() try: yield db finally:
db.close()
```

La variable DATABASE_URL utilise le format sqlite:///./clinique.db, ce qui indique à SQLAlchemy d'utiliser SQLite et de créer le fichier clinique.db dans le répertoire courant du projet. SQLite a l'avantage de ne nécessiter aucun serveur de base de données séparé — c'est un simple fichier sur le disque, idéal pour un projet académique.

Le paramètre connect_args={"check_same_thread": False} est obligatoire pour SQLite avec FastAPI. Par défaut, SQLite interdit à plusieurs threads d'utiliser la même connexion simultanément. Mais FastAPI étant asynchrone, il peut traiter plusieurs requêtes en parallèle. Ce paramètre désactive cette restriction.

SessionLocal est configurée avec autocommit=False, ce qui signifie que les changements ne sont pas envoyés automatiquement à la base — il faut appeler db.commit() explicitement. autoflush=False évite les flush automatiques non désirés. La fonction get_db() est un générateur Python : elle ouvre une session, la yield (la fournit à l'endpoint), puis la ferme dans le bloc finally, même en cas d'erreur.

```
(venv) PS C:\Users\styves\Desktop\clinique>
(venv) PS C:\Users\styves\Desktop\clinique> python -c "from database import Base, get_db; print('database.py OK')"
database.py OK
(venv) PS C:\Users\styves\Desktop\clinique> |
```

Figure 6 — Vérification de database.py : "database.py OK" confirme le bon fonctionnement

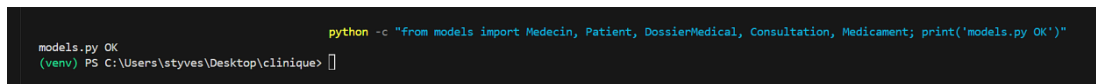
4. MODÉLISATION DES DONNÉES — models.py

C'est le fichier le plus important du projet, et celui qui nous a pris le plus de temps. Il définit la structure complète de la base de données sous forme de classes Python. Chaque classe représente une table, chaque attribut représente une colonne.

4.1 La table pivot Many-to-Many

La première chose définie dans models.py est la table pivot `consultation_medicament`. Contrairement aux autres entités, cette table n'est pas une classe Python mais une instance de `Table()`. Elle contient deux colonnes : `consultation_id` et `medicament_id`, qui forment ensemble la clé primaire composite. Cela garantit qu'une même paire consultation-médicament ne peut apparaître qu'une seule fois.

```
consultation_medicament = Table( "consultation_medicament", Base.metadata,
    Column("consultation_id", Integer, ForeignKey("consultations.id"),
    primary_key=True), Column("medicament_id", Integer, ForeignKey("medicaments.id"),
    primary_key=True) )
```



```
python -c "from models import Medecin, Patient, DossierMedical, Consultation, Medicament; print('models.py OK')"
```

models.py OK
(venv) PS C:\Users\styves\Desktop\clinique> █

Figure 7 — Imports SQLAlchemy et définition de la table pivot Many-to-Many

4.2 Les modèles Medecin, Patient, DossierMedical

La classe `Medecin` est la plus simple : quatre colonnes (`id`, `nom`, `prenom`, `specialite`) et une relation One-to-Many vers `Consultation`. La classe `Patient` ressemble à `Medecin` mais inclut deux relations : une vers `DossierMedical` et une vers `Consultation`.

Le point crucial du `Patient` c'est le paramètre `uselist=False` dans le `relationship()` vers `DossierMedical`. Sans ce paramètre, SQLAlchemy retournerait une liste — même si elle ne contient qu'un seul élément. Avec `uselist=False`, il retourne directement l'objet, ce qui correspond à la sémantique du One-to-One.

`DossierMedical` contient les informations médicales du patient (groupe sanguin, antécédents, allergies). Sa clé étrangère `patient_id` est définie avec `unique=True`, ce qui est le mécanisme côté base de données qui enforces le One-to-One : SQLite refusera physiquement d'insérer deux dossiers avec le même `patient_id`.

4.3 Les modèles Consultation et Medicament

`Consultation` est la classe centrale car elle est reliée à trois autres tables. Elle contient `patient_id` et `medecin_id` comme clés étrangères, et déclare trois relations : vers `Patient`, vers `Medecin`, et vers `Medicament` via `secondary=consultation_medicament` pour le Many-to-Many.


```
PS C:\Users\styves\Desktop\clinique> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\styves\\Desktop\\clinique']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [14940] using StatReload
INFO: Started server process [7296]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Figure 8 — Vérification models.py OK et lancement du serveur uvicorn

5. SCHÉMAS DE VALIDATION — schemas.py

Pydantic est la bibliothèque qui gère la validation des données dans FastAPI. schemas.py définit les "contrats" de l'API : ce qu'elle accepte en entrée et ce qu'elle retourne en sortie. Ce fichier est totalement indépendant de la base de données.

Pour chaque entité, nous avons défini trois classes. XxxBase contient les champs communs. XxxCreate hérite de XxxBase et représente ce que le client envoie lors d'un POST ou PUT — sans l'id, car c'est SQLite qui le génère. XxxOut hérite aussi de XxxBase mais ajoute l'id et les données imbriquées des relations pour les réponses GET.

La classe Config avec from_attributes = True est indispensable dans tous les schémas Out. SQLAlchemy retourne des objets Python, pas des dictionnaires. Pydantic, par défaut, attend des dictionnaires. Ce paramètre lui dit : "accepte aussi les objets avec attributs".

```
class PatientOut(PatientBase): id: int dossier: Optional[DossierOut] = None
consultations: List[ConsultOut] = [] class Config: from_attributes = True
```

Optional[DossierOut] signifie que le champ peut être None — si le patient n'a pas encore de dossier. C'est un objet unique (pas une liste) ce qui reflète le One-to-One. List[ConsultOut] est une liste qui peut être vide, ce qui reflète le One-to-Many. Et chaque ConsultOut contient elle-même une List[MedicamentOut] pour les données Many-to-Many — trois niveaux d'imbrication.

6. IMPLÉMENTATION DES ENDPOINTS — main.py

main.py est le fichier le plus long du projet. Il crée l'instance FastAPI, initialise les tables au démarrage, et définit les 22 endpoints. Deux lignes critiques au début :

```
app = FastAPI(title="API Clinique Medicale")
models.Base.metadata.create_all(bind=engine)
```

La deuxième ligne parcourt tous les modèles enregistrés dans Base.metadata et crée les tables correspondantes dans clinique.db si elles n'existent pas encore. C'est automatique — pas besoin de migrations manuelles.

6.1 Endpoints Médecins

Cinq endpoints gèrent les médecins. GET /medecins retourne tous les médecins avec db.query(models.Medecin).all(). POST /medecins crée un médecin en recevant un MedecinCreate validé par Pydantic, crée un objet SQLAlchemy, le sauvegarde avec db.add() + db.commit(), et utilise db.refresh() pour récupérer l'id généré. PUT /medecins/{id} cherche le médecin, retourne 404 s'il n'existe pas, puis modifie ses attributs et appelle db.commit(). DELETE suit la même logique.

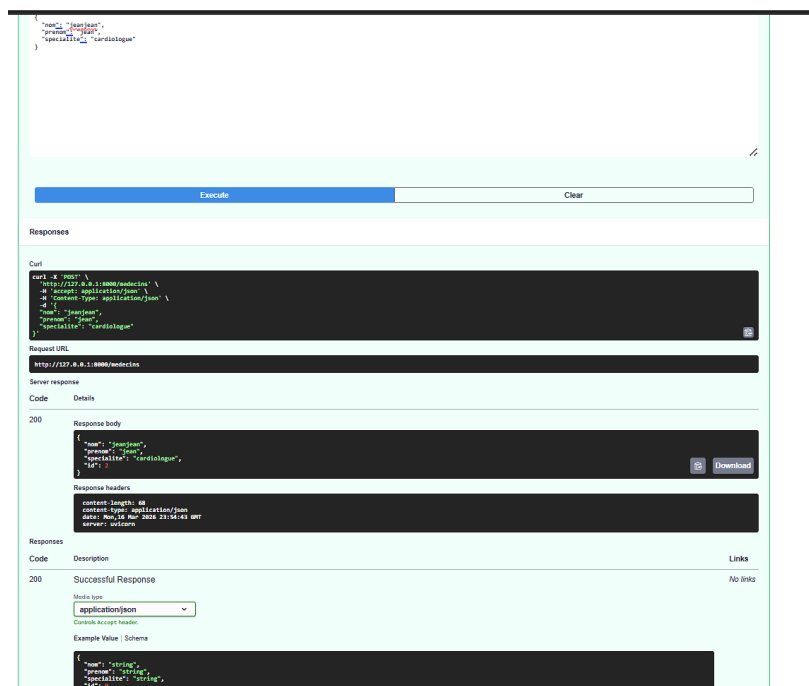


Figure 9 — POST /medecins : création du Dr. Jeanjean jean, Cardiologie (Code 200)

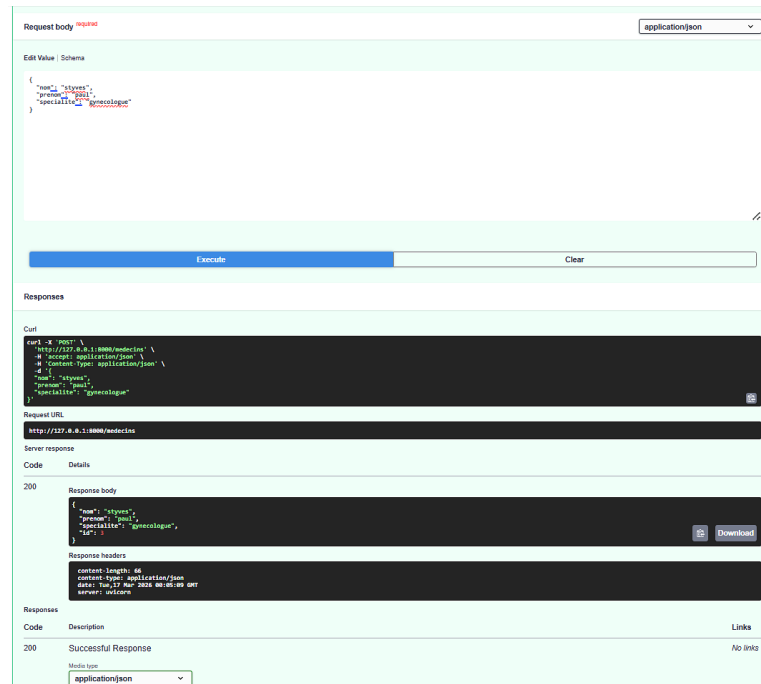


Figure 10 — POST /medecins : création du Dr. Styves PAUL, Gynécologie (id auto: 3)

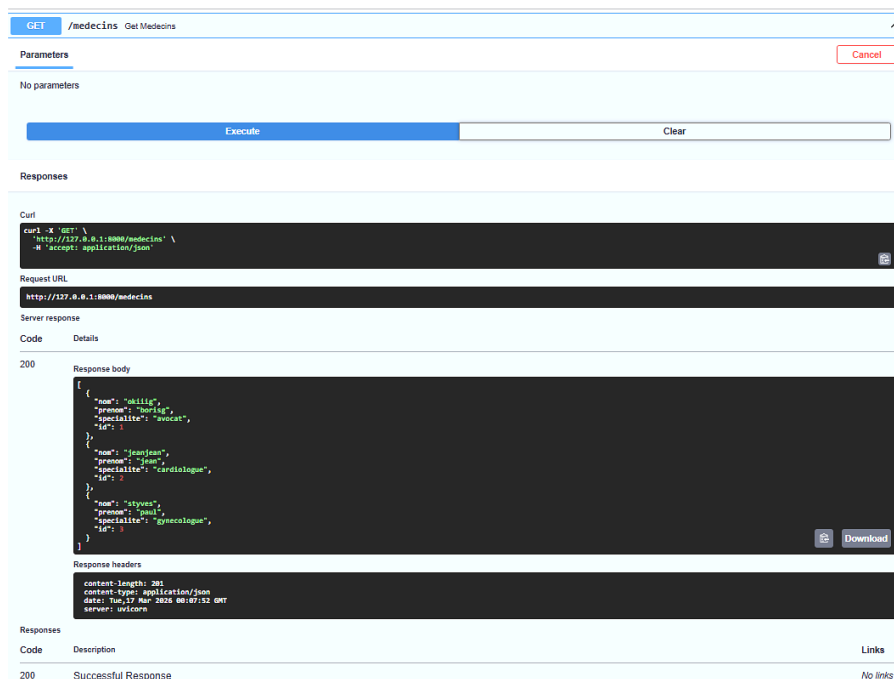


Figure 11 — GET /medecins : liste complète des 3 médecins enregistrés

6.2 Endpoints Patients et Dossiers

GET /patients/{id} est le endpoint le plus intéressant pour les patients. En retournant simplement l'objet patient avec le schéma PatientOut, FastAPI charge automatiquement le dossier médical associé grâce aux relations SQLAlchemy. Pas de jointure à écrire — c'est la magie de l'ORM.

POST /dossiers contient la logique la plus délicate. Deux vérifications sont effectuées : le patient doit exister (404 sinon), et il ne doit pas déjà avoir un dossier (400 sinon). Cette double protection garantit l'intégrité du One-to-One.

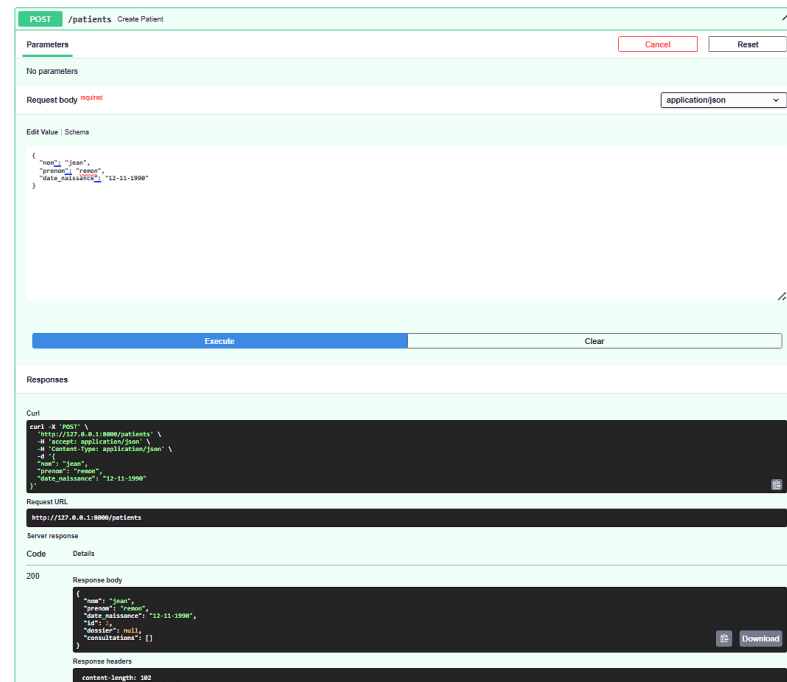


Figure 12 — POST /patients : Jean Remon créé, dossier=null, consultations=[]

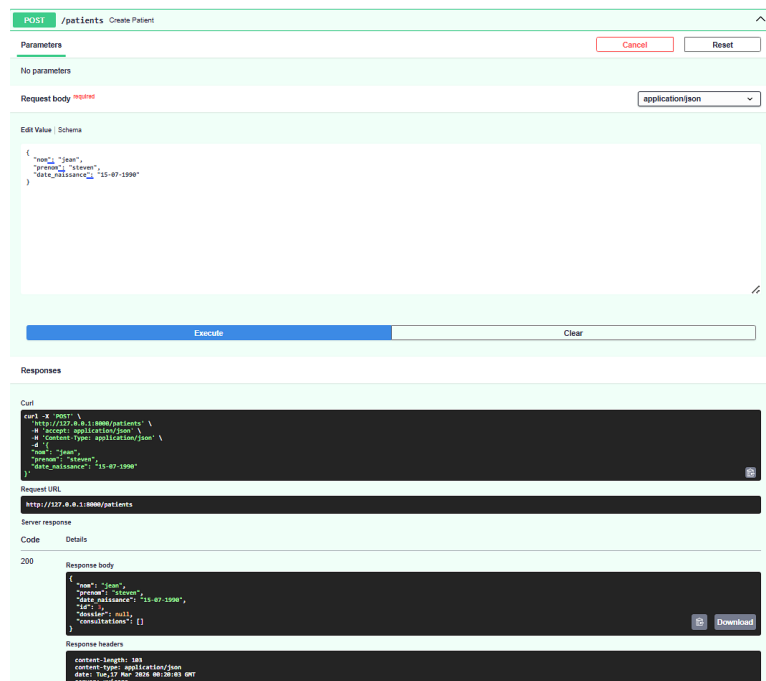


Figure 13 — POST /patients : JEAN Steven, dossier null, consultations vides

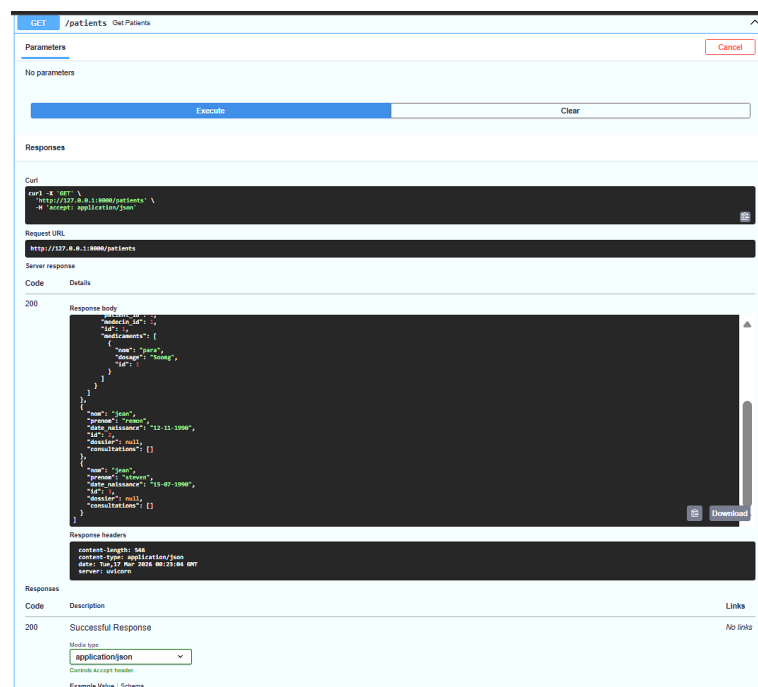


Figure 14 — GET /patients : liste complète avec données imbriquées

6.3 Endpoints Consultations

POST /consultations vérifie que le patient et le médecin existent avant de créer la consultation. GET /consultations/{id} retourne la consultation avec sa liste de médicaments prescrits imbriquée — SQLAlchemy les charge via la table pivot automatiquement.

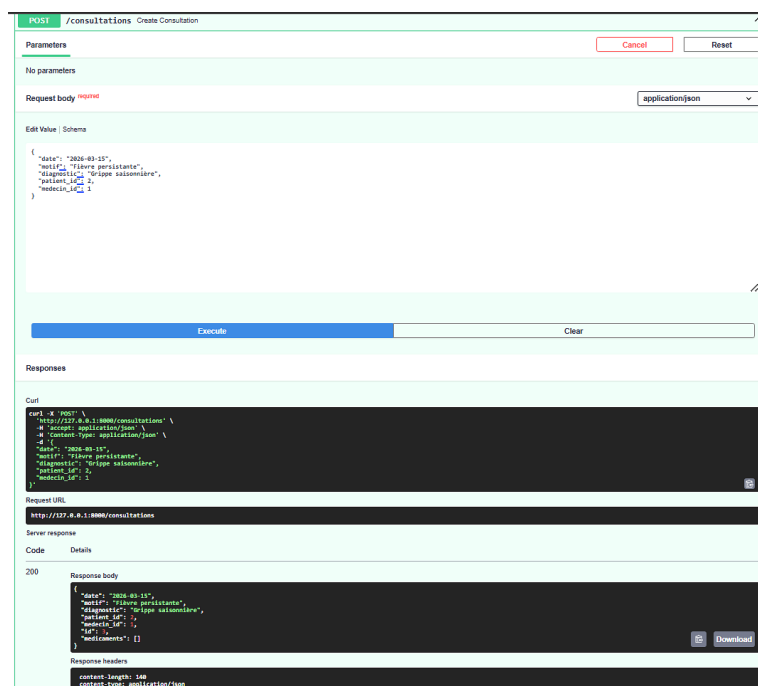


Figure 15 — POST /consultations : création réussie, médicaments=[] vide

6.4 Endpoints Prescriptions et Médicaments

POST /prescriptions est le endpoint le plus complexe. Après trois vérifications (consultation existe, médicament existe, pas déjà prescrit), la prescription est créée avec `consultation.medicaments.append(medicament)`. SQLAlchemy traduit ça en INSERT INTO `consultation_medicament` sans qu'on écrive une seule ligne SQL.

The screenshot shows a REST client interface for a POST request to `/consultations`. The request body is a JSON object with the following structure:

```
{
  "date": "18-2-2007",
  "medic": "yous",
  "diagnostic": "vous te maig",
  "patient_id": 1,
  "medecin_id": 2
}
```

The response is a 200 status code with a JSON body:

```
{
  "date": "18-2-2007",
  "medic": "yous",
  "diagnostic": "vous te maig",
  "patient_id": 1,
  "medecin_id": 2,
  "medicaments": []
}
```

Figure 16 — GET /medicaments : liste des 3 médicaments avec id, nom, dosage

7. IMPLÉMENTATION DES RELATIONS SQLALCHEMY

7.1 One-to-One — Patient et DossierMedical

Trois mécanismes travaillent ensemble pour garantir cette relation. Au niveau SQLite, `unique=True` sur `patient_id` dans `dossiers_medicaux` empêche physiquement les doublons. Au niveau Python, `uselist=False` dans le `relationship()` retourne un objet unique. Au niveau API, le contrôle applicatif dans `POST /dossiers` retourne 400 si un dossier existe déjà.

The screenshot displays a REST client interface with two tabs. The first tab, titled `GET /medicaments`, shows a successful response with a status code of 200. The response body is a JSON object: `{ "nom": "Parac", "dosage": "500mg", "se": 1 }`. The second tab shows a 400 error response, likely from a `POST /dossiers` request, indicating a conflict or invalid data. The response body is a JSON object: `{ "nom": "string", "dosage": "string", "se": 0 }`. The interface includes fields for the request URL, parameters, and response details.

Figure 17 — Erreur 400 : tentative de créer un second dossier pour le même patient

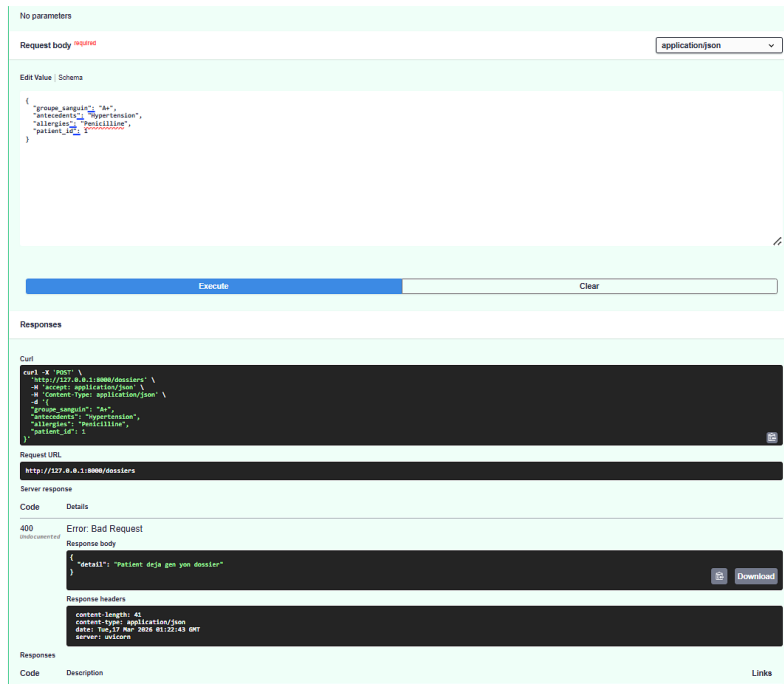


Figure 18 — POST /dossiers réussi : dossier créé avec id automatique

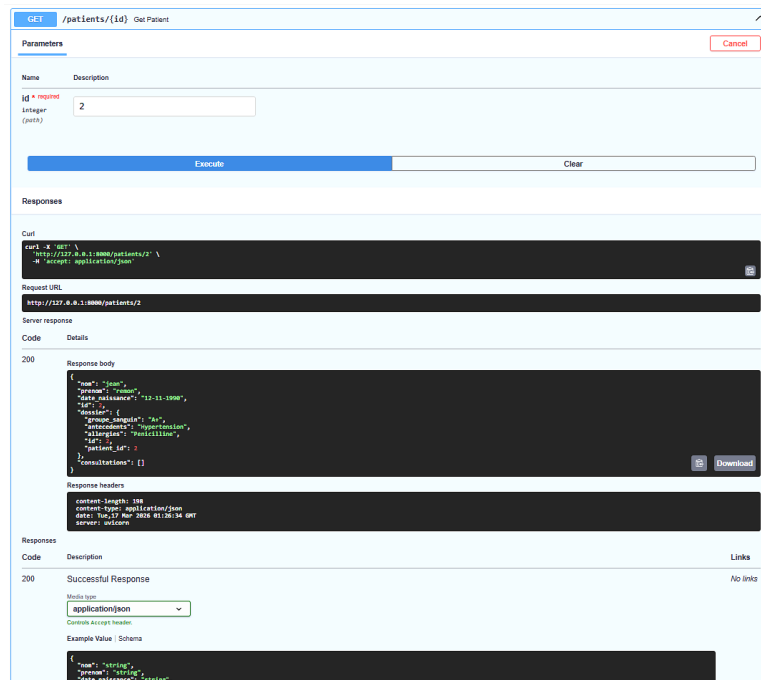


Figure 19 — GET /patients/{id} : patient retourné avec dossier médical imbriqué (One-to-One)

7.2 One-to-Many — Medecin/Patient vers Consultation

La règle fondamentale est que la clé étrangère se place toujours du côté "Many". Donc Consultation possède patient_id et medecin_id, pas l'inverse. Les relationship() avec back_populates assurent la cohérence bidirectionnelle : modifier un côté met à jour l'autre.

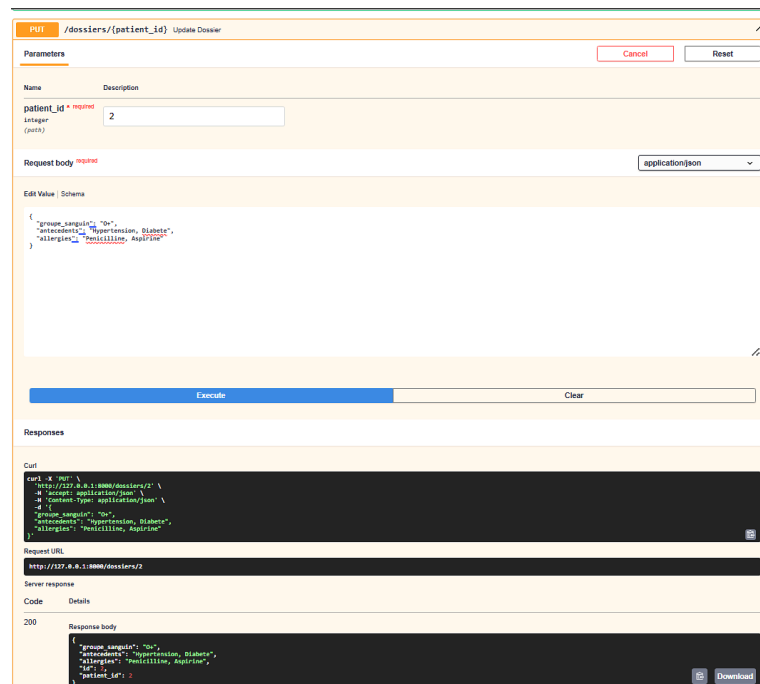


Figure 20 — PUT /dossiers : mise à jour du dossier, groupe sanguin A+ → O+

7.3 Many-to-Many — Consultation et Medicament

C'est la relation la plus complexe. La table pivot stocke les paires (consultation_id, médicament_id). Le paramètre secondary= dans les relationship() des deux côtés indique à SQLAlchemy d'utiliser cette table intermédiaire. Toutes les opérations SQL sur le pivot sont générées automatiquement.

```
# Ajouter une prescription (INSERT dans pivot)
consultation.medicaments.append(medicament) db.commit() # Retirer une prescription
(DELETE dans pivot) consultation.medicaments.remove(medicament) db.commit() #
Vérifier doublon if medicament in consultation.medicaments: raise
HTTPException(status_code=400)
```

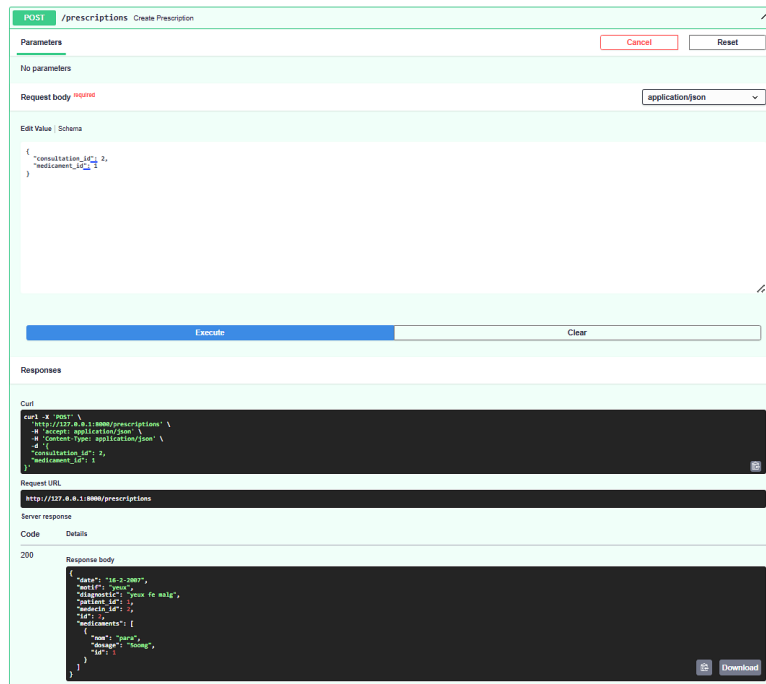


Figure 21 — POST /prescriptions : Paracétamol prescrit, médicaments mis à jour

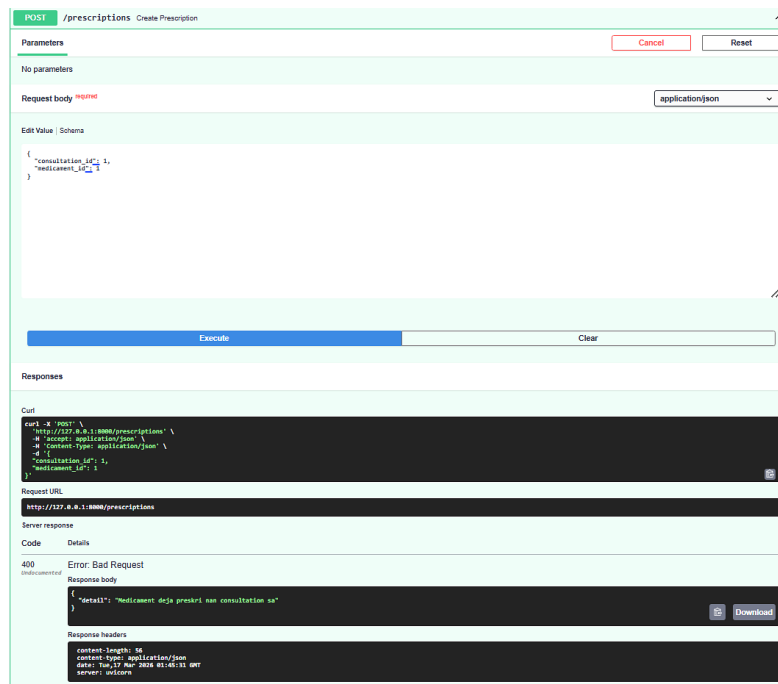


Figure 22 — Erreur 400 : médicament déjà prescrit dans cette consultation

8. GESTION DES ERREURS HTTP

Trois codes d'erreur sont gérés dans l'API. Le 404 est retourné lorsqu'une ressource n'existe pas en base. Systématiquement, chaque endpoint qui reçoit un id vérifie si l'objet existe avec `.first()`, et lève `HTTPException(status_code=404)` si le résultat est `None`. Par exemple : `GET /patients/999` retourne `{"detail": "Patient non trouve"}` si aucun patient avec cet id n'existe.

Le 400 Bad Request est différent : la requête est syntaxiquement correcte, mais elle viole une règle métier. Deux cas dans notre projet. Premier cas : créer un dossier pour un patient qui en a déjà un (violation One-to-One). Deuxième cas : prescrire un médicament déjà prescrit dans la même consultation (doubleton Many-to-Many).

Le 422 Unprocessable Entity est géré automatiquement par Pydantic, sans qu'on écrive de code spécifique. Si un client envoie un POST sans le champ obligatoire "nom", Pydantic retourne instantanément une réponse 422 détaillée indiquant quel champ manque et pourquoi.

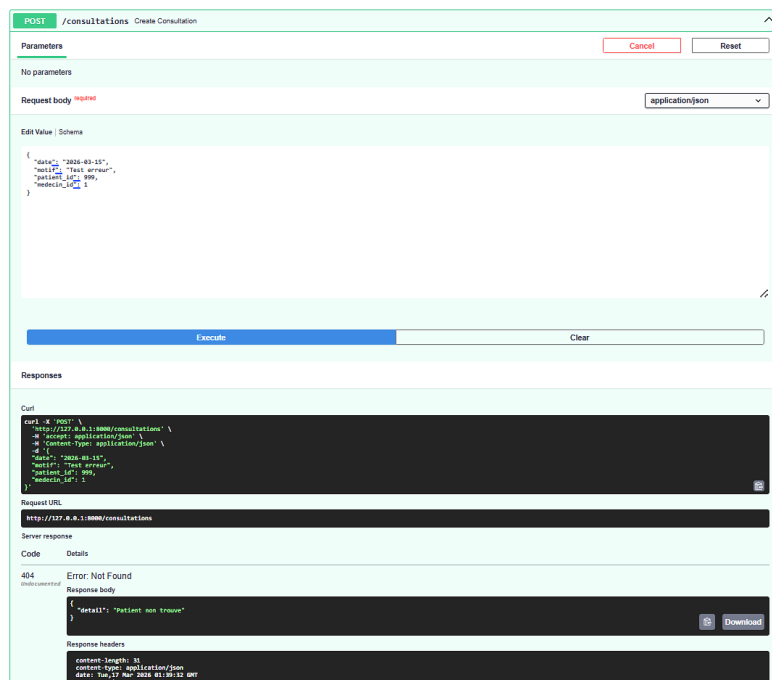


Figure 23 — Erreur 404 : consultation avec patient_id=999 inexistant

DELETE

/prescriptions/{cid}/{mid} Delete Prescription

Parameters

Cancel

Name	Description
cid * required integer (path)	4
mid * required integer (path)	4

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8080/prescriptions/4/4' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8080/prescriptions/4/4

Server response

Code	Details
404	Error: Not Found

Response body

```
{
  "detail": "Consultation non trouve"
}
```

Download

Response headers

```
content-length: 36
content-type: application/json
date: Tue, 17 Nov 2020 11:06:15 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Content Accept header

Example Value | Schema

Figure 24 — Erreur 404 : DELETE /prescriptions avec consultation inexistante

9. TESTS ET VALIDATION VIA SWAGGER UI

Tous les tests ont été réalisés sur `http://127.0.0.1:8000/docs`, l'interface Swagger UI générée automatiquement par FastAPI. Pour chaque endpoint, nous avons cliqué "Try it out", saisi les données, exécuté la requête et vérifié le code de réponse et le JSON retourné.

Pour les médecins, nous avons créé trois médecins avec `POST /medecins` (codes 200, ids auto-générés 1, 2, 3), puis vérifié leur présence avec `GET /medecins` (liste complète retournée). Pour les patients, deux patients ont été créés — les champs dossier et consultations étaient bien null et [] à ce stade, comme attendu.

Le test le plus intéressant a été `GET /patients/{id}` après création d'un dossier : la réponse incluait le dossier médical imbriqué automatiquement. C'est la démonstration concrète que la relation One-to-One fonctionnait correctement.

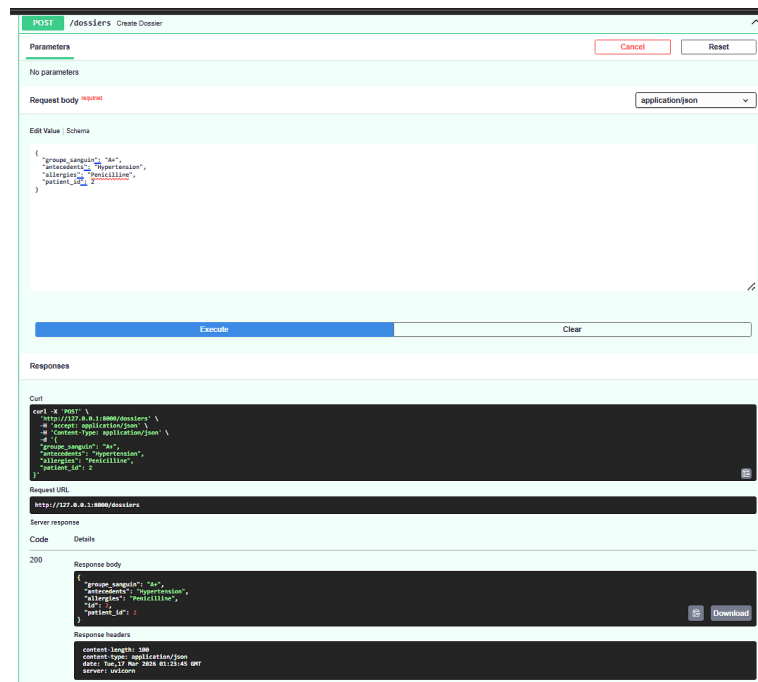


Figure 25 — `POST /dossiers` : préparation création dossier patient 2

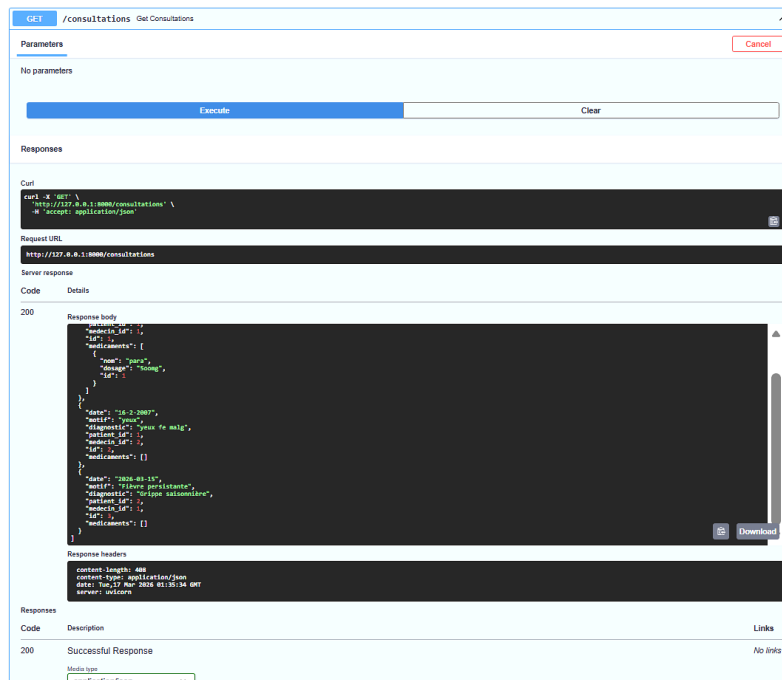


Figure 26 — GET /consultations : liste avec médicaments imbriqués (Many-to-Many)

Pour les prescriptions, nous avons d'abord prescrit le Paracétamol dans la consultation 2 (code 200, médicaments mis à jour dans la réponse). Ensuite, une tentative de prescrire le même médicament une deuxième fois a bien retourné une erreur 400 "Médicament déjà preskri nae consultation sa". Le DELETE /prescriptions avec une consultation inexistante a retourné 404.

10. DIFFICULTÉS RENCONTRÉES ET SOLUTIONS APPORTÉES

On ne va pas mentir — ce projet nous a donné du fil à retordre. Voici les problèmes qu'on a vraiment rencontrés, dans l'ordre où ils se sont présentés.

La première erreur SQLAlchemy :

Au tout début, quand on a essayé de lancer le serveur pour la première fois, SQLAlchemy a affiché "Mapper could not assemble any primary key columns for mapped table patients". On a cherché pendant un moment avant de réaliser que Integer manquait dans la ligne d'import. Sans Integer, le `primary_key=True` ne fonctionnait pas. Une fois Integer ajouté, tout s'est débloqué.

Le DNS qui pointait vers le routeur :

Pour la connexion de la machine cliente au domaine Active Directory, on a passé un bon moment à se demander pourquoi `ping travay.local` retournait "could not find host". En examinant `ipconfig /all`, on a vu que le DNS préféré pointait vers le routeur (192.168.10.254) au lieu du serveur AD (192.168.10.1). Le routeur ne connaît pas la zone DNS privée `travay.local`. Correction : modifier manuellement le DNS préféré vers l'IP du serveur AD. Après ça, ça marchait.

Le decorator @ oublié :

`POST /patients` n'apparaissait pas dans Swagger UI alors qu'on l'avait pourtant écrit dans `main.py`. On a relu le fichier en entier avant de remarquer que `app.post()` était écrit sans le `@` au début. Sans `@`, Python l'interprète comme un simple appel de fonction, pas comme un decorator. FastAPI ne l'enregistre donc pas comme route. Une lettre manquante, vingt minutes de débogage.

Le response_model incorrect pour les médicaments :

`POST /medicaments` ne retournait pas l'id dans sa réponse, ce qui nous empêchait de faire les prescriptions puisqu'on avait besoin de `medicament_id`. En regardant le code, on a vu que `response_model=schemas.MedicamentCreate` était utilisé au lieu de `schemas.MedicamentOut`. `MedicamentCreate` n'a pas d'id (c'est le schéma d'entrée). `MedicamentOut` l'a. Un simple changement, un vrai soulagement.

11. CONCLUSION

Ce projet a été plus formateur qu'on ne le pensait au départ. Au début, on voyait FastAPI comme "juste un autre framework". Mais en travaillant dessus, on a compris pourquoi il est devenu aussi populaire dans l'industrie. La génération automatique de Swagger UI, l'injection de dépendances, l'intégration native avec Pydantic — c'est vraiment bien pensé.

Ce qui nous a le plus marqués, c'est SQLAlchemy. L'idée de définir des relations en Python et de laisser l'ORM générer le SQL correspondant, ça change vraiment la façon de travailler. On n'a pas écrit une seule requête SQL dans tout le projet, et pourtant la base de données fait exactement ce qu'on voulait.

Individuellement, chacun a gagné en compétences. Comprendre la différence entre One-to-One, One-to-Many et Many-to-Many, c'est une chose. Les implémenter correctement avec les bons paramètres (`uselist=False`, `unique=True`, `secondary=`), c'en est une autre. On pense pouvoir dire qu'on maîtrise maintenant ces concepts.

Pour l'avenir, ce système pourrait évoluer avec un système d'authentification JWT pour sécuriser les endpoints, une migration vers PostgreSQL pour la production, des tests unitaires avec pytest, et peut-être un frontend en React pour avoir une interface graphique complète. Pour l'instant, on est satisfaits du résultat obtenu.